Counting Sentences Lab (CodeGrade)

(https://github.com/learn-co-curriculum/python-p3-oo-counting-sentences-lab) (https://github.com/learn-co-curriculum/python-p3-oo-counting-sentences-lab/issues/new)

Learning Goals

- Practice defining properties and instance methods on a class.
- Practice defining instance methods that use the `self` keyword.

Key Vocab

- **Class**: a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
- **Initialize**: create a working copy of a class using its `__init__` method.
- **Instance**: one specific working copy of a class. It is created when a class's `__init__` method is called.
- **Object**: the more common name for an instance. The two can usually be used interchangeably.
- **Object-Oriented Programming**: programming that is oriented around data (made mobile and changeable in **objects**) rather than functionality. Python is an object-oriented programming language.
- **Function**: a series of steps that create, transform, and move data.
- **Method**: a function that is defined inside of a class.
- **Magic Method**: a special type of method in Python that starts and ends with double underscores. These methods are called on objects under certain conditions without needing to use their names explicitly. Also called **dunder methods** (for **d**ouble **under**score).
- **Attribute**: variables that belong to an object.
- **Property**: attributes that are controlled by methods.

Introduction

As was discussed in an earlier lesson, Python includes a number of built in methods that can be used with different data types. To see those methods, we can use the `dir()` method. For example, if we want to see all the built in methods belonging to the `String` class, we can run the following:

```
$ dir(str)
# => ['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
```

Note that this will also work if we pass any *instance* of the `String` class as the argument - give it a try!

But what if we needed some additional functionality for string objects? One way we could do that is to create our own class — `MyString`, say — and write the methods ourselves. As long as the value of an instance of our `MyString` class is a string, we could not only call any methods we create ourselves on the instance, but also call any of the built in string methods on its value!

Instructions

For this lab, you will be creating the `MyString` class and several methods on the class. You will need to use the `self` keyword in the body of these methods to refer to the instance of `MyString` on which the method is being called.

`MyString`

Create the `MyString` class and give it a `value` *property*. The class should verify that the `value` is a string before assigning it. The default value of `value` should be the empty string, `''`.

`is_sentence()`

Define an instance method `is_sentence()` that returns `True` if the value ends in a **period** and `False` if it does not.

Hint: You might want to take a look at the list of built in string methods above to see if there's something there that can help you.

`is_question()`

This method should return `True` if the value ends with a **question mark** and `False` if it does not.

`is_exclamation()`

This method should return `True` if the value ends with an **exclamation mark** and `False` if it does not.

`count_sentences()`

What we'd like to be able to do is call a `count_sentences()` method on a `MyString` instance, and get back a, well, count of sentences in its `value`. In other words:

```
string = MyString()
string.value = "This is a string! It has three sentences. Right?"
string.count_sentences()
# => 3
```

This is a tricky task in any language, but Python provides us a few tools to streamline the process:

- `str.replace(old, new)` (https://docs.python.org/3/library/stdtypes.html?highlight=replace#str.replace) will replace any instances of `old` in `str` with `new`.
- `str.split(pattern)` (https://docs.python.org/3/library/stdtypes.html?highlight=split#str.split) will split a string into a list using the provided `pattern` as the separator.
- The re (https://docs.python.org/3/library/re.html) module (covered later on in this phase's optional Regular Expressions module) will allow you to search for multiple patterns at once. If you're feeling bold, check out the linked documentation and give it a shot!

Remember to consider edge cases, such as the following sentence:

```
This, well, is a sentence. This is too!! And so is this, I think? Woo...
```

What would happen if we split this sentence on the punctuation characters? We would end up with a list that contains empty strings as well as strings containing sentences. How would you eliminate empty strings from a list?

Resources

- **String `replace()` method (https://docs.python.org/3/library/stdtypes.html?highlight=replace#str.replace)**
- **String `split()` method (https://docs.python.org/3/library/stdtypes.html?highlight=split#str.split)**
- **re module (https://docs.python.org/3/library/re.html)**

This tool needs to be loaded in a new browser window

Load Counting Sentences Lab (CodeGrade) in a new window